

PROGRESS REPORT

Project Details

Project: Tower of Hanoi

Duration: 7 Days

Language: Python

Team Members

1. Isaiah Andrei Noda
2. Hannali Olayvar

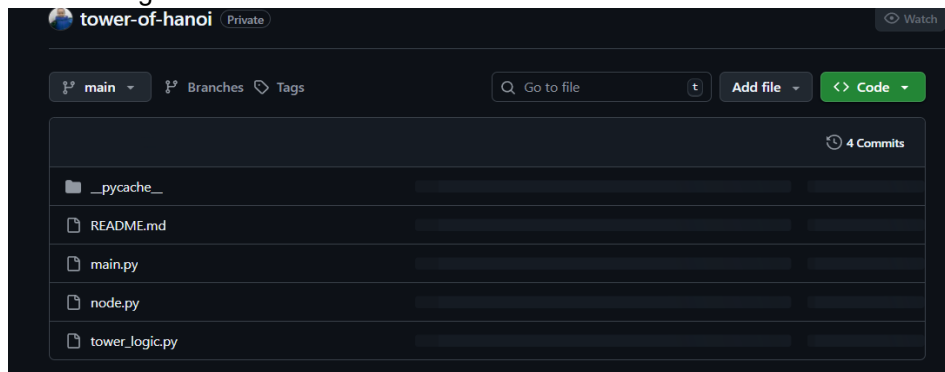
Development Process Overview

October 4, 2025 – Planning and Setup

Objective: Discuss setup on how to collaborate

Actions Taken:

- Created a GitHub repository for version control and remote collaboration.
- Selected Live Share extension in Visual Studio Code for collaborative coding.
- Initialized and cloned the repository locally for each team member.
- Discussed git controls

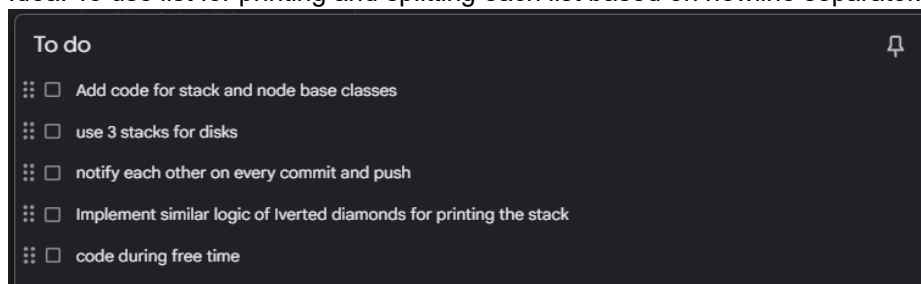


October 5, 2025 – Initial Design and Logic Planning

Objective: Finalize implementation logic and plan data representation.

Ideas Made:

- Idea: Initially considered storing numerical values in stack and converting them to asterisks during display.
- Idea: To use list for printing and splitting each list based on newline separator.



October 6, 2025 – Coding and Implementation

Objective: Implement code for logic without stack first (we used stack immediately)

Actions Taken:

- Reused Stack and Node classes from tower logic module.
- Created three main stacks (A, B, C) instances.
- Created initializeStacks() to create towers.
 - Before inserting any disks, the function ensures that labels A, B, and C are pushed at the bottom of each stack.
 - For Tower A, the function pushes all disks based on the entered number of disks, using a formula to calculate spacing and alignment.
 - For Towers B and C, blank lines are pushed first as spacers/fillers to maintain equal stack height and structure.
- Created display () to print stacks side-by-side with stacks centered. Lists were not used, and opted to use stack directly to lessen memory usage. Stored asterisks directly instead of numbers to lessen code.
- Added validation for user disk input (3–10).
- Used Live Share for 3-hour joint coding session.

```
14     print("Enter a number between 3-10.")
15     except ValueError:
16         print("Enter an Integer.")
17
18     initializeStacks(disknum) #initializes stacks
19     #printStacks()
20     display()
21
22     print("\nGame start! Move all disks from A to C.")
23     print("Type 'X' to exit. ")
24
25     while True:
26         move = input("Enter move (e.g., A C): ").strip().upper()
27         if move == "X":
28             print("Exited successfully")
29             break
30
31         #splitting the move into two separate parts
32         splittedmove = move.split()
33         if len(splittedmove) != 2:
34             print("Please enter only two letters.")
35         if splittedmove[0] not in "abc" or splittedmove[1] not in "abc":
36             print("Please enter from ABC only genuine format A C not AC")
37
38
39     def display():
40         current = disk1.top
41         currenttt = disk2.top
42         currentttt = disk3.top
43         while current and currenttt and currentttt:
44             print(current.value, currenttt.value, currentttt.value)
45             current = current.next
46             currenttt = currenttt.next
47             currentttt = currentttt.next
```

October 15, 2025 – Coding and Implementation Part 2

Objective: Implement code for logic of moving towers

Actions Taken:

- Modified code for initializing stacks, wherein .center() method is used to center the elements instead of calculating the spaces manually
- Created function moveDiskAgain with parameters source, destination, and maxDisk. Current implementation fails. Logic is not correct and always turns out to be an INVALID MOVE.
- Used Live Share for 2-hour joint coding session.

```
70 def moveDiskAgain(source, destination, maxDisk):
71     s = {
72         "A": disk1,
73         "B": disk2,
74         "C": disk3
75     }
76
77     src = s[source]
78     dst = s[destination]
79
80     if len(str(src.peek()).strip()) > len(str(dst.peek()).strip()):
81
82         disk = src.pop().strip()
83         lines = -1
84         while not dst.is_empty():
85             current = dst.peek()
86             if "A" in str(current).strip() or "B" in str(current).strip() or "C" in str(current).strip():
87                 break
88             if str(current).strip() == "|":
89                 lines += 1
90             dst.pop()
91
92         #for disk and spacers n/a
93         maxwidth = maxDisk * 2 + 1
94         dst.push(disk.center(maxwidth))
95
96         #for spacers
97         for i in range(lines):
98             spacer = "|".center(maxwidth)
99             dst.push(spacer)
100
101
```

October 16, 2025 – Coding and Final Debugging Part 3

Objective: Fix code logic and Debug errors

Actions Taken:

- Created new function `getLastDisk()` that takes the stack as a parameter and traverses through it to find the last valid element.
 - Returns 1 if the last element is a letter or line and Returns the specific disk if it's a disk.
- Updated invalid move validation:
 - If `getLastDisk()` returns ("|") or if the source value length is less than the destination value length the move is allowed.
- Modified traversal and insertion algorithms for both source and destination stacks for moving:
 - Traverses the source to locate and pop the correct disk.
 - Traverses the destination to insert the disk at the correct point (before the base line or letter).
- Avoided use of external data structures (like lists or temporary stacks) to maintain pure stack logic.
- Debugged issue where extra lines were pushed:
 - Cause found: same variable used for line counting in both source and destination.
 - Fixed by assigning separate counters for each.
- Confirmed that disk movement logic now works properly and output remains centered.
- Used Live Share for 3-hour joint coding session

```
def getLastDisk(stack):
    lines = 0
    maxwidth = maxDisk * 2 + 3
    spacer = "|".center(maxwidth)

    while not stack.is_empty():
        current = stack.peak()
        if "A" in str(current).strip() or "B" in str(current).strip() or "C" in str(current).strip() or "*" in str(current).strip():
            nahanapnadisk = str(current).strip()
            break
        if str(current).strip() == "|":
            lines += 1
            stack.pop()

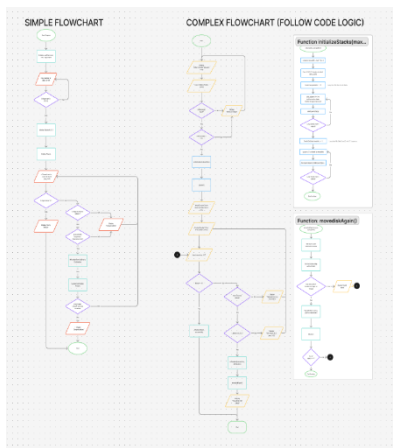
    #for spacers
    for i in range(lines):
        stack.push(spacer)
    lines = 0
    return nahanapnadisk
```

October 17, 2025 – Documentation and Flowcharting

Objective: Document classes and create program flowcharts for the Tower of Hanoi project

Actions Taken:

- Worked from 9:00 PM to 12:00 AM (3 hours total) on google docs and FigJam
- Created and formatted the documentation table & Added code snippets and comments for clarity.
- Documented the Node, Stack, and Main classes with their attributes and methods.
- Started and worked on the flowcharts and Made two versions:
 - Simple flowchart which shows main program logic.
 - Detailed flowchart which includes symbols and shows each function, loop, and validation.
- Reviewed each other's work for accuracy.



Classes	
• Class Node in node.py	
Attribute	Description
value	Stores the data in the data field
next	Points to the next node in the stack
Description: Initialization of a single element(nodes) in the linked list to be used later in the stack	
• Class Stack in stack.py	
Method	Description
__init__	Initializes an empty stack and sets the top to 'None'
push(value)	Adds a new item to the top of the stack
pop()	Removes the top of stack and returns its value
is_empty()	Checks if the stack is empty (no disks)
peek()	Returns top disk value without deleting
Description: This class initializes and manages stack objects used to represent the towers in the Tower of Hanoi game. We can perform the above mentioned stack operations for handling stack data structure using linked lists.	
• Main script in main.py	

Method	Description
main()	The main function that starts the program. It asks the user for the number of disks (3-10), initializes the towers, displays them, and handles user inputs for moving disks between towers.
display()	Prints the current state of the three towers (A, B, and C) side by side, showing how the disks are arranged on each tower.
initializeStacks(maxDisk)	Set up the initial state of the game. Tower A is filled with all disks represented by " " while Towers B and C are initialized with spacers " ".
getLastDisk(stack)	Retrieves the topmost disk from a given stack without permanently removing it. It is used to check which disk is currently on top before validating or performing a move.
moveDiskAgg(source, destination)	Handles the logic of moving a disk from one tower to another. It checks if the move is valid (no larger disk can be placed on a smaller one) and updates both source and destination towers accordingly. Displays the new tower state after each move.

October 18, 2025 – Finalization of Document

Objective: Check documentation and finalize reference links

Actions Taken:

- Worked from 8:30 PM to 10:00 PM on google docs
- Fixed and formatted the documentation
- Reformatted the classes and added the two flowcharts in the documentation
- Added reference links to the document

Technical Specifications

Main Components:

- Three Stack instances (disk1, disk2, disk3) represent towers A, B, and C.
- Main class handles program flow.
- Node class contains structure of a Node,
- Stack class contains structure of a class and operations.
- *initializeStacks()* fills the stacks with disks represented by strings of asterisks (*).
- *display()* prints the towers side by side
- *getLastDisk(stack)* traverses through the stack and gets last Disk for validation purposes
- *moveDiskAgain(source, destination)* moves the disk from source to destination

Tools and Environment

GitHub - Version control and remote collaboration.

Visual Studio Code - IDE

(Live Share) - Real-time collaborative coding Extension.

Python 3 - Programming language used.

Discord Communications - Main communication mode during live share sessions